**SIMATS ENGINEERING**

**Department of Information Technology**

**DSA01 – OBJECT-ORIENTED PROGRAMMING WITH C++**

**ASSIGNMENT 3 – CO3**

# Constructors, Destructors and Operator Overloading

| | | | |
|---|---|---|---|
| **Student Name** | _Pavithra Sri S__ | **Register Number** | __192572079_____ |
| **Department / Section** | _CSE with AI___ | **Academic Year / Semester** | ___2nd yr___ |
| **Faculty Name** | __Amirtha_____ | **Nature of Assignment** | Individual |
| **Duration** | 1 Week | **Total Marks** | 50 |
| **Date of Submission** | _____ | **Date of Evaluation** | _____ |
| **GitHub Repository** | DSA01-<RegisterNumber>-Assignment3 | **Compiler** | g++ (C++17) |

**GitHub Link: _____**

*CO3 Coverage: Constructors, destructors, dynamic memory, copy control and operator overloading.*

**MARK SHEET (Rubric-based Evaluation)**

| Task | Task Title | Max Marks | Marks Awarded | Remarks |
|---|---|---|---|---|
| Q1 | Constructors and Constructor Overloading | 6 | | |
| Q2 | Copy Constructors: Shallow and Deep Copies | 7 | | |
| Q3 | Dynamic Constructors and Destructors | 7 | | |
| Q4 | Arithmetic and Stream Operator Overloading | 7 | | |
| Q5 | Increment, Comparison and Subscript Operators | 6 | | |
| Q6 | Rules of Operator Overloading and Text Class | 7 | | |
| Q7 | Complex-Number Class Design | 10 | | |
| | **TOTAL** | **50** | | |

OVERALL REMARKS

Student Signature Faculty Signature Program Director

**ASSIGNMENT 3 – CO3**

**CO3: Construct C++ programs using constructors, destructors and operator overloading to control object creation, destruction and operator behaviour.**

Nature: Individual | Duration: 1 Week | Total Marks: 50 | Bloom's Level: K5 – Evaluate

**Mandatory Execution and GitHub Submission**

- Create a repository named DSA01-<RegisterNumber>-Assignment3 and share it with the faculty.
- Create one folder for each task (Q1 to Q7), containing source code, input files and output screenshots.
- Add a README.md with name, register number, compiler/version, exact compile command, run steps and a short result for every task.
- Commit progressively, with at least one commit per task. Include sanitizer reports and measured timings where requested.
- Submit the GitHub link on the cover page. Do not invent execution timings or sanitizer results; fill them after running the programs.

GitHub Repository Link: _____

**How to Complete the Assignment**

Complete the Code: Replace each TODO with working code without changing unrelated parts of the supplied program, then compile and compare the output.

Predict the Output: Write the predicted output before running the program. Explain any difference using the relevant C++ rule.

Complexity and Evidence: Compare alternatives using Big-O notation. Add actual measurements, compiler output and sanitizer evidence after execution.

*Note: The solutions below provide implementations and expected results. Run each program locally with g++ -std=c++17 -Wall -Wextra and attach your own screenshots and measurements.*

**Q1 – Constructors and Constructor Overloading (6 Marks)**

**1. Design and Implementation**

A Book record stores a title and a non-negative number of copies. A default record is empty, a donated book may provide only a title, and a complete record provides both title and copies. Constructor delegation keeps validation in one place.

```
#include <iostream>
#include <stdexcept>
#include <string>
class Book {
    std::string title;
    int copies;
public:
    Book() : Book("", 0) {}
    explicit Book(const std::string& t) : Book(t, 0) {}
    Book(const std::string& t, int c) : title(t), copies(c) {
        if (c < 0) throw std::invalid_argument("Copies cannot be
negative");
    }
    void print() const {
        std::cout << "Title: " << (title.empty() ? "(empty)" :
title)
                  << ", Copies: " << copies << '\n';
    }
};
int main() {
    Book empty;
    Book donated("C++ Basics");
    Book full("Data Structures", 12);
    empty.print(); donated.print(); full.print();
    try { Book invalid("Invalid Book", -3); }
    catch (const std::invalid_argument& e) { std::cout << e.what()
<< '\n'; }
}
```

**2. Expected Output**

```
Title: (empty), Copies: 0
Title: C++ Basics, Copies: 0
Title: Data Structures, Copies: 12
Copies cannot be negative
```

**3. Design and Analysis**

- The compiler supplies a default constructor only when no user-declared constructor exists. Once any constructor is declared, an implicit default constructor is not generated.
- Member initializer lists directly initialise members. Assignment inside the constructor body first default-constructs members and then assigns values.
- Members are initialised in the order they are declared in the class, not the order written in the initializer list.

- State after construction: empty = ("", 0); donated = ("C++ Basics", 0); full = ("Data Structures", 12).
- The title-only constructor is explicit so a string cannot be silently converted to a Book. Without explicit, `Book b = std::string("Novel");` compiles; with explicit, copy-initialisation is rejected. Direct initialisation `Book b("Novel");` still works.

**4. Experimental Investigation**

Removing Book() makes `Book books[3];` fail because each array element requires a default constructor. A negative-copy test throws invalid_argument. To observe -Wreorder, deliberately initialise members in a different order from their declaration; compile with `-Wreorder` and the compiler warns that actual initialisation follows declaration order.

**5. Alternative: One Constructor with Default Arguments**

```
Book(std::string t = "", int c = 0) : title(std::move(t)), copies(c) {
    if (c < 0) throw std::invalid_argument("Copies cannot be negative");
}
```

Both designs create an object in O(1) time and use O(1) extra space. A single constructor centralises validation in one location, while overloads make the three creation situations clearer at the call site. Constructor delegation also centralises validation in the two-argument constructor.

**Q2 – Copy Constructors: Shallow and Deep Copies (7 Marks)**

**1. Problem and Memory Behaviour**

A shallow copy copies the pointer value. The original and copied Prescription then refer to the same medicine array. Changing one changes the other, and both destructors attempt to delete the same memory, causing a double-free error.

A deep copy allocates a separate array and copies every medicine name. The two objects then have independent storage.

Shallow memory: p.medicines ──┐ and q.medicines ──┴──> same array. Deep memory: p.medicines ──> Array A; q.medicines ──> Array B.

**2. Completed Code (Deep Copy)**

```
#include <iostream>
#include <string>
class Prescription {
    int n;
    std::string* medicines;
public:
    explicit Prescription(int count)
        : n(count), medicines(count > 0 ? new std::string[count] :
nullptr) {}

    Prescription(const Prescription& other)
        : n(other.n),
          medicines(other.n > 0 ? new std::string[other.n] : nullptr) {
        for (int i = 0; i < n; ++i) medicines[i] = other.medicines[i];
    }

    Prescription& operator=(const Prescription& other) {
        if (this == &other) return *this;
        std::string* fresh =
            other.n > 0 ? new std::string[other.n] : nullptr;
        for (int i = 0; i < other.n; ++i) fresh[i] = other.medicines[i];
        delete[] medicines;
        medicines = fresh;
        n = other.n;
        return *this;
    }

    ~Prescription() { delete[] medicines; }

    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};
int main() {
    Prescription p(2);
    p.set(0, "Paracetamol"); p.set(1, "ORS");
    Prescription q = p;
```

```
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p;
    p.set(0, "Ibuprofen");
    std::cout << p.get(0) << ' ' << p.get(1) << " | "
              << q.get(0) << ' ' << q.get(1) << " | "
              << r.get(0) << ' ' << r.get(1) << '\n';
}
```

**3. Expected Output**

```
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
```

**4. Design and Analysis**

- Rule of Three: if a class manages a resource and needs a destructor, it usually needs a copy constructor and copy assignment operator too.
- Copy constructor is used when creating an object from another object, passing an object by value, and returning an object by value when copy elision does not remove the copy.
- Passing by const reference avoids making a copy. Passing by value invokes a copy for an lvalue argument.
- Self-assignment `p = p` is safe because the assignment operator checks `this == &other`.
- The compiler-generated copy constructor is suitable when members already manage their own copying safely, such as std::string and std::vector. A custom copy is required for raw owning pointers.

**5. Alternative: std::vector**

```
#include <vector>
#include <string>
struct PrescriptionVector {
    std::vector<std::string> medicines;
};
```

The vector copy constructor performs an independent element copy automatically (Rule of Zero). Copying n medicine names takes O(n) time and O(n) additional space for both a vector-based design and the correct deep-copy array design. A shallow pointer copy is O(1), but is unsafe because it aliases ownership.

**6. Experimental Evidence to Record**

Compile both versions using `g++ -std=c++17 -Wall -Wextra -fsanitize=address -g q2.cpp -o q2`. Run with `ASAN_OPTIONS=detect_leaks=1 ./q2`. Record the shallow version's double-free/use-after-free report and the deep-copy version's clean run. Measure copy times for 10, 1,000 and 100,000 medicines; do not fill timings until measured.

**Q3 – Dynamic Constructors and Destructors (7 Marks)**

**1. Dynamic Seat Map Class**

```
#include <iostream>
#include <string>
class Event {
    int rows, seats;
    char** map;
    int booked = 0;
public:
    Event(int r, int s) : rows(r), seats(s), map(nullptr) {
        if (rows < 0 || seats < 0) throw std::invalid_argument("Invalid
dimensions");
        if (rows > 0) {
            map = new char*[rows]{};
            for (int i = 0; i < rows; ++i) {
                map[i] = new char[seats];
                for (int j = 0; j < seats; ++j) map[i][j] = 'O';
            }
        }
        std::cout << "Created " << rows << "x" << seats << '\n';
    }
    bool book(int r, int s) {
        if (r < 0 || r >= rows || s < 0 || s >= seats || map[r][s] ==
'X')
            return false;
        map[r][s] = 'X'; ++booked; return true;
    }
    bool cancel(int r, int s) {
        if (r < 0 || r >= rows || s < 0 || s >= seats || map[r][s] ==
'O')
            return false;
        map[r][s] = 'O'; --booked; return true;
    }
    ~Event() {
        for (int i = 0; i < rows; ++i) delete[] map[i];
        delete[] map;
        std::cout << "Closed event; booked seats = " << booked << '\n';
    }
    Event(const Event&) = delete;
    Event& operator=(const Event&) = delete;
};
int main() {
    Event stackEvent(2, 3);
    stackEvent.book(0, 1);
    {
        Event blockEvent(1, 2);
        blockEvent.book(0, 0);
    }
```

```
    Event* heapEvent = new Event(2, 2);
    heapEvent->book(1, 1);
    delete heapEvent;
    Event emptyEvent(0, 5);
}
```

*Note: Add `#include <stdexcept>` to the source file for invalid_argument.*

**2. Expected Lifecycle Order**

```
Created 2x3
Created 1x2
Closed event; booked seats = 1
Created 2x2
Closed event; booked seats = 1
Created 0x5
Closed event; booked seats = 0
Closed event; booked seats = 1
```

The block-scope object is destroyed when its block ends. The heap object is destroyed only when delete is executed. Remaining local objects are destroyed in reverse order of construction when main ends.

**3. Design and Analysis**

- The memory layout is an array of row pointers; each pointer refers to a separately allocated row of seats. Total seat storage is rows × seats bytes, plus rows pointers.
- Forgetting delete leaks the object and its arrays. Using delete instead of delete[] for an array is undefined behaviour.
- A destructor has no parameters and no return type, and a class can have only one destructor; therefore it cannot be overloaded.
- RAII ties resource release to object lifetime. When an object leaves scope or is deleted, its destructor releases owned memory automatically.

**4. Alternative Representations**

```
// One contiguous block:
char* seats = new char[rows * cols];
seats[r * cols + c] = 'O';

// Or:
std::vector<std::vector<char>> seats(
    rows, std::vector<char>(cols, 'O'));
```

Both provide O(1) indexed access. A contiguous block uses one allocation and no row-pointer array; the row-pointer design uses rows + 1 allocations and O(rows) pointer overhead. A vector-of-vectors uses a vector allocation for the outer container plus row allocations, but manages cleanup automatically.

**5. Experimental Evidence**

Use AddressSanitizer to compare a deliberately leaked heap event with the corrected delete path. Approximate seat-map payload sizes for char storage: 10×20 = 200 bytes; 100×200 = 20,000 bytes; 1,000×2,000 = 2,000,000 bytes, excluding allocator and pointer overhead. A zero-row event should construct and destruct safely.

**Q4 – Overloading Arithmetic and Stream Operators (7 Marks)**

**1. Money Class and Completed TODOs**

```
#include <iomanip>
#include <iostream>
#include <cstdlib>
class Money {
    long long paise;
public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p)
{}
    Money operator+(const Money& other) const {
        return Money(0, static_cast<int>(paise + other.paise));
    }
    bool operator==(const Money& other) const { return paise ==
other.paise; }
    friend std::ostream& operator<<(std::ostream& os, const Money&
m) {
        long long value = m.paise;
        if (value < 0) { os << "-Rs "; value = -value; }
        else os << "Rs ";
        return os << value / 100 << '.' << std::setw(2) <<
std::setfill('0')
                  << value % 100 << std::setfill(' ');
    }
};
int main() {
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    std::cout << c << '\n' << std::boolalpha << (c == Money(200)) <<
'\n';
    Money carry(0, 75), extra(0, 50);
    std::cout << carry + extra << '\n';
}
```

For a production-quality class, store the total paise directly through a private factory or add a constructor that accepts total paise. The short constructor above is adequate for the given positive demonstration values; avoid converting very large totals through int.

**2. Expected Output**

```
Rs 200.00
true
Rs 1.25
```

**3. Operator Design**

- The + operator returns a new Money value and does not modify either operand. The += operator should modify the left operand and return Money& (`return *this`) to support chaining.
- Member operators are suitable for +, ==, < and += because the left operand is a Money object. Stream insertion << must be non-member because its left operand is std::ostream.
- A friend << function can access private paise while still allowing `std::cout << money`.

- The expression `a + b` is translated to `a.operator+(b)` for a member overload, or `operator+(a, b)` for a non-member overload.
- Subtraction can produce a negative amount; this implementation's formatting convention displays a leading minus sign. A billing application may instead reject negative balances based on business rules.
- Multiplying Money by Money is not meaningful for ordinary currency arithmetic. Multiplying Money by a numeric quantity or scalar can be meaningful.

**4. Alternative: Non-member Operators and Named Functions**

```
Money operator+(const Money& a, const Money& b) {
    return Money::fromPaise(a.totalPaise() + b.totalPaise());
}
bool equals(const Money& a, const Money& b) {
    return a.totalPaise() == b.totalPaise();
}
```

Member and non-member versions take O(1) time and O(1) extra space. A non-member overload can support symmetric expressions such as `100 + m` when an implicit conversion from 100 is allowed; a member overload cannot be called on the built-in left operand.

*Note: For the final submitted implementation, use a dedicated total-paise constructor/factory and include -, +=, < and >> if the full implementation tasks are being demonstrated. The supplied code completes the two TODOs and the given expected output.*

**Q5 – Increment, Comparison and Subscript Operators (6 Marks)**

**1. Time and Playlist Implementation**

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
class Time {
   int seconds;
public:
   explicit Time(int s = 0) : seconds(s) {}
   Time& operator++() { ++seconds; return *this; }       // prefix
   Time operator++(int) { Time old(*this); ++seconds; return old; } // postfix
   bool operator==(const Time& t) const { return seconds == t.seconds; }
   bool operator<(const Time& t) const { return seconds < t.seconds; }
   void print() const {
      int h = seconds / 3600, m = (seconds % 3600) / 60, s = seconds % 60;
      std::cout << h << ':' << (m < 10 ? "0" : "") << m
                << ':' << (s < 10 ? "0" : "") << s;
   }
};
class Playlist {
   std::vector<std::string> tracks;
public:
   explicit Playlist(std::vector<std::string> t) : tracks(std::move(t)) {}
   std::string& operator[](std::size_t i) {
      if (i >= tracks.size()) throw std::out_of_range("Track index out of range");
      return tracks[i];
   }
   const std::string& operator[](std::size_t i) const {
      if (i >= tracks.size()) throw std::out_of_range("Track index out of range");
      return tracks[i];
   }
};
int main() {
   Time t(3599);
   Time a = t++;
   Time b = ++t;
   std::cout << "a="; a.print(); std::cout << " b="; b.print();
   std::cout << " t="; t.print(); std::cout << '\n';
   Playlist p({"Track One", "Track Two"});
   std::cout << p[1] << '\n';
}
```

**2. Predicted Output for the Given Counter Program**

5 7 7

Initially c.v = 5. Postfix c++ returns a copy of the old value 5, then c becomes 6. Prefix ++c increments c to 7 and returns the updated object, so b.v and c.v are both 7. Postfix makes a copy; prefix returns a reference.

### 3. Design and Analysis

- The compiler distinguishes postfix ++ by its dummy int parameter: operator++() is prefix and operator++(int) is postfix.
- Postfix returns the old value, so it must create a copy. Prefix changes the object and returns *this by reference.
- A const and non-const operator[] are needed so read-only playlists can be indexed while mutable playlists can modify a track.
- The sample uses std::out_of_range for invalid indexes such as -1 or size. The unsigned index parameter means a negative integer converts to a large unsigned value and is rejected by the bounds check.
- The Time example uses total seconds, so 59:59 advances to 1:00:00. For an application with a 24-hour clock, explicitly apply modulo 86,400.

### 4. Alternative: at() or Unchecked Access

std::vector::at(i) checks bounds and throws std::out_of_range. Unchecked operator[] does not perform a range check and invalid access has undefined behaviour. All are O(1) per valid access; checked access adds one comparison and a branch. Measure 10,000,000 accesses at -O0 and -O2 to report the actual cost.

**Q6 – The Rules of Operator Overloading (7 Marks)**

**1. Text Class Using Dynamic Memory**

```
#include <cstring>
#include <iostream>
class Text {
   char* data;
public:
   Text(const char* s = "") {
      data = new char[std::strlen(s) + 1];
      std::strcpy(data, s);
   }
   Text(const Text& other) {
      data = new char[std::strlen(other.data) + 1];
      std::strcpy(data, other.data);
   }
   Text& operator=(const Text& other) {
      if (this == &other) return *this;
      char* fresh = new char[std::strlen(other.data) + 1];
      std::strcpy(fresh, other.data);
      delete[] data;
      data = fresh;
      return *this;
   }
   ~Text() { delete[] data; }
   Text operator+(const Text& other) const {
      std::size_t n = std::strlen(data) + std::strlen(other.data);
      char* temp = new char[n + 1];
      std::strcpy(temp, data);
      std::strcat(temp, other.data);
      Text result(temp);
      delete[] temp;
      return result;
   }
   bool operator==(const Text& other) const {
      return std::strcmp(data, other.data) == 0;
   }
   bool operator!=(const Text& other) const { return !(*this == other); }
   friend std::ostream& operator<<(std::ostream& os, const Text& t) {
      return os << t.data;
   }
};
int main() {
   Text a("Hello "), b("World");
   Text c = a + b;
   std::cout << c << '\n';
   Text empty;
   Text copy = empty;
   a = a; // self-assignment test
   std::cout << std::boolalpha << (empty == copy) << '\n';
```

```
}
```

**2. Expected Output**

```
Hello World
true
```

**3. Rules of Operator Overloading**

- Operators that cannot be overloaded include ::, ., .*, ?: and sizeof.
- Overloading cannot change an operator's precedence, associativity or number of operands (arity). At least one operand must be a user-defined type.
- operator= must be a non-static member function.
- A friend is useful when an operator needs private access but its left operand is not the class, such as stream insertion `std::cout << text`.
- The Rule of Three applies because Text owns a raw character array: destructor, copy constructor and copy assignment must work together.
- For `a + b + c`, the expression is left-associative: `(a + b) + c`. Each + creates a combined result. Count actual copy-constructor calls by adding a diagnostic message to the copy constructor and running the program.

**4. Alternative: std::string or += on a Copy**

```
std::string result = a + b + c;

// Alternative operator+ implementation:
Text operator+(Text left, const Text& right) {
    left += right;
    return left;
}
```

std::string manages memory and copying automatically, so it avoids manual Rule-of-Three code. For strings of lengths n1, n2 and n3, a left-associated `a+b+c` copies approximately n1+n2 characters into the first temporary and then n1+n2+n3 into the next result: O(n1+n2+n3) character work, with temporary storage. An +=-on-a-copy implementation can reuse the left temporary's buffer and reduce some allocations, but still requires copying the initial left operand. Compare with std::string using 100,000 concatenations and AddressSanitizer.

**Q7 – Design a Complex-Number Class from First Principles (10 Marks)**

**Stage 1 – Requirements**

- Data: real part and imaginary part, both stored as double.
- Operations: addition, subtraction, multiplication, division, equality, rectangular printing and magnitude.
- Constructors: default (0 + 0j), real-only and real-plus-imaginary.
- Measurable requirements: each arithmetic operation returns the mathematically correct result; division by zero is detected; output shows rectangular form and magnitude.

**Stage 2 – Class Design and Formulae**

For z1 = a + bj and z2 = c + dj: addition = (a+c) + (b+d)j; subtraction = (a-c) + (b-d)j; multiplication = (ac-bd) + (ad+bc)j; division = $((ac+bd)/(c^2+d^2)) + ((bc-ad)/(c^2+d^2))j$. Magnitude = $\text{sqrt}(a^2+b^2)$.

**Stage 3 – Implementation and Verification**

```
#include <cmath>
#include <iostream>
#include <stdexcept>
class Complex {
   double re, im;
public:
   Complex() : re(0), im(0) {}
   Complex(double real) : re(real), im(0) {}
   Complex(double real, double imag) : re(real), im(imag) {}

   Complex operator+(const Complex& z) const {
      return Complex(re + z.re, im + z.im);
   }
   Complex operator-(const Complex& z) const {
      return Complex(re - z.re, im - z.im);
   }
   Complex operator*(const Complex& z) const {
      return Complex(re*z.re - im*z.im, re*z.im + im*z.re);
   }
   Complex operator/(const Complex& z) const {
      double d = z.re*z.re + z.im*z.im;
      if (d == 0.0) throw std::domain_error("Division by zero impedance");
      return Complex((re*z.re + im*z.im)/d,
                     (im*z.re - re*z.im)/d);
   }
   bool operator==(const Complex& z) const {
      const double eps = 1e-9;
      return std::abs(re-z.re) < eps && std::abs(im-z.im) < eps;
   }
   double magnitude() const { return std::sqrt(re*re + im*im); }
   friend std::ostream& operator<<(std::ostream& os, const Complex& z) {
      os << z.re << (z.im < 0 ? " - " : " + ")
         << std::abs(z.im) << 'j';
      return os;
   }
```

```
};
int main() {
   try {
      // Series RLC: R = 10 ohm, XL = +j5, XC = -j10
      Complex R(10), L(0, 5), C(0, -10);
      Complex series = R + L + C;
      std::cout << "Series Z = " << series
            << ", |Z| = " << series.magnitude() << '\n';

      // Parallel branches: 10 ohm resistor in parallel with j10 ohm inductor
      Complex resistor(10), inductor(0, 10);
      Complex parallel = (resistor * inductor) / (resistor + inductor);
      std::cout << "Parallel Z = " << parallel
            << ", |Z| = " << parallel.magnitude() << '\n';

      Complex zero;
      Complex invalid = resistor / zero;
      std::cout << invalid << '\n';
   } catch (const std::domain_error& e) {
      std::cout << e.what() << '\n';
   }
}
```

**4. Hand Verification and Expected Output**

Series circuit: $Z = 10 + j5 - j10 = 10 - j5$. Magnitude = $\sqrt{(10^2 + (-5)^2)} = \sqrt{125} \approx 11.1803\ \Omega$.

Parallel circuit: $Z = (10 \times j10)/(10 + j10) = j100/(10+j10) = 5+j5$. Magnitude = $\sqrt{(5^2+5^2)} = \sqrt{50} \approx 7.0711\ \Omega$.

```
Series Z = 10 - 5j, |Z| = 11.1803
Parallel Z = 5 + 5j, |Z| = 7.07107
Division by zero impedance
```

**5. Stage 4 – Defend the Design**

- The default constructor represents zero impedance. The real-only constructor represents a pure resistance. The two-part constructor represents general complex impedance.
- Arithmetic and comparison operators make circuit expressions readable. The stream operator is a friend because the left operand is an ostream.
- std::complex<double> already provides tested complex arithmetic and is preferred for normal applications. A custom class is useful when learning operator overloading or enforcing domain-specific rules and formatting.
- A limitation is exact floating-point equality: the epsilon comparison is only a simple tolerance policy and may need scaling for values of very different magnitudes.

**6. Alternative: std::complex<double>**

```
#include <complex>
std::complex<double> z1(10, 5), z2(0, -10);
auto series = z1 + z2;
auto parallel = (std::complex<double>(10,0) *
                 std::complex<double>(0,10)) /
                (std::complex<double>(10,0) +
                 std::complex<double>(0,10));
```

Both custom Complex and std::complex<double> use O(1) time per arithmetic operation and O(1) space (two double values). std::complex is standardised and well tested. Division by zero still needs an explicit application-level policy if the program must report an error rather than propagate non-finite floating-point values.

**FINAL SUBMISSION CHECKLIST**

- Create DSA01-<RegisterNumber>-Assignment3 on GitHub.
- Upload Q1–Q7 source files into separate folders.
- Compile using g++ -std=c++17 -Wall -Wextra (and -fsanitize=address for memory-management tasks).
- Add screenshots of actual program outputs and sanitizer reports.
- Write output predictions before execution for Q3 and Q5.
- Record actual copy timings, memory measurements and benchmark results; do not use estimated values as measured results.
- Add README.md with compiler/version, commands, run instructions and short task-wise results.
- Add the working GitHub link to the cover page and submit the report.

**End of Assignment 3 – CO3**